实验 1　操作系统实验环境安装与 Shell 命令

一、实验目标

(1)了解 openEuler 基础知识

(2)掌握 openEuler 安装过程

(3)掌握 openEuler 基本命令

二、背景知识

2.1 openEuler 系统

华为 openEuler 是一款开源的操作系统。openEuler 内核源于 Linux，支持鲲鹏及其它多种处理器。它能充分释放计算芯片潜能，由全球开源贡献者构建的高效、稳定、安全的开源操作系统，适用于数据库、大数据、云计算、人工智能等应用场景。同时，openEuler 是一个面向全球的操作系统开源社区，通过社区合作，打造创新平台，构建支持多处理器架构、统一和开放的操作系统，推动软硬件应用生态繁荣发展。

openEuler 关键特性如下：

(1)iSula 轻量级容器解决方案，统一 IoT、边缘和云计算容器解决方案：

①缩短三级调用链，容器内存资源占用相比 Docker 引擎显著下降。

②支持 CRI/OCI 标准开源接口，灵活对接 runc、kata 等多种 OCI 运行时。

③通过 Smart-loading 智能镜像下载技术，显著提升镜像下载速度。

④安全容器：虚拟化技术和容器技术的有机结合，安全容器具有更好的隔离性。

⑤系统容器：支持本地文件系统启动，可实现快速部署。支持部署 systemd，提升 user namespace 隔离性。

(2)Kunpeng 加速引擎（KAE），支持加解密加速：

①摘要算法 SM3，支持异步模型。

②对称加密算法 SM4，支持异步模型，支持 CTR/XTS/CBC 模式。

③对称加密算法 AES，支持异步模型，支持 ECB/CTR/XTS/CBC 模式。

④非对称算法 RSA，支持异步模型，支持 Key Sizes 1024/2048/3072/4096。

⑤密钥协商算法 DH，支持异步模型，支持 Key Sizes 768/1024/1536/2048/3072/4096。

(3)A-Tune 智能系统性能优化引擎，推理出业务特征，配置最佳的系统参数合，使业务处

于最优运行状态。

(4)增强 glibc/zlib/gzip 性能，充分利用 AArch64 的 neon 指令集，提升基础库性能。

(5)内核特性增强：

①支持 ARM64 内核热补丁。

②Numa Aware Qspinlock：减少跨 NUMA 节点的 Cache/总线冲突。

③通过优化 IOVA 页表查找和页表释放算法，提升 I/O MMU 子系统性能。

④根据 ARM64 指令以及流水线特点，优化 CRC32 及 checksum 实现，大幅提升数据校验性能。

⑤支持 ARM v8.4 MPAM（Memory System Resource Partitioning and Monitoring）。

(6)虚拟化特性增强：

①中断虚拟化优化：IRQfd 路径注入中断优化，大幅提升高性能直通设备（网卡、SSD 磁盘等）性能。

②内存虚拟化优化：借助鲲鹏硬件特性，提升虚拟机启动内存加载速度。

③存储虚拟化优化：iSCSI 模块 kworker 的 NUMA 亲和性自绑定优化，提升 IPSAN 磁盘的 IO 性能。

有关 openEuler 操作系统的更多内容，详见其官网：https://openeuler.org/zh/，本实验手册不再逐一阐述。

2.2 FinalShell 软件

FinalShell 是一款服务器、网络管理软件，不仅是 ssh 客户端，还是功能强大的开发、运维工具，充分满足开发、运维需求。该软件的主要特性包括以下几个方面：

(1)支持多种协议，包括 SSH、Telnet、SFTP 等，可连接不同操作系统。

(2)具有自动重连、多标签页、多窗口、快捷键、脚本支持、FTP/SFTP 文件传输、X11 转发等丰富的功能。

(3)界面友好，支持自定义主题、字体、颜色等，满足个性化需求。

(4)支持插件机制，可以通过插件扩展软件的功能。

(5)支持自定义配置，用户可以根据自己的需求调整软件的外观和功能。

(6)能够快速连接、管理和操作远程服务器，适用于各种规模的 IT 环境。

(7)具有稳定性和安全性，保障用户数据的安全和可靠性。

2.3 常用 Shell 命令

(1)什么是 Shell?

Shell 是一种命令行解释器，主要用于在操作系统中运行命令和编写的脚本。Shell 可以解释用户输入的命令，并将命令发送给操作系统进行执行。

(2)Shell 的功能有哪些?

①解释命令：Shell 可以解释用户输入的命令，并将命令发送给操作系统进行执行。这使得用户可以通过 Shell 与操作系统进行交互，执行各种任务，如管理文件、运行程序、操作系统设置等。

②管理进程：Shell 可以启动和停止进程，也可以通过 Shell 命令来管理正在运行的进程。例如，ps 命令可以查看正在运行的进程列表，kill 命令可以终止某个进程的运行。

③管理文件系统：Shell 可以通过各种命令来管理文件系统，例如创建、复制、删除文件或目录，查找文件，以及对文件进行权限管理等。

④环境变量管理：Shell 使用环境变量来保存各种配置信息，如 PATH 变量保存可执行文件的搜索路径，HOME 变量保存当前用户的主目录等。Shell 命令可以用来管理和配置这些环境变量。

⑤脚本编写：除了直接在命令行中使用 Shell 命令外，还可以编写 Shell 脚本来自动化一些任务。Shell 脚本是一系列 Shell 命令的集合，可以按照一定的逻辑结构自动化执行各种任务。

(3)Shell 的种类有哪些？

常见的 Shell 包括 Bash、Zsh、Ksh、Csh 等。不同 Shell 的语法和功能略有差异，但基本的 Shell 命令和功能都是相同的。

(4)Shell 命令的基本语法

Shell 命令通常由命令名、参数和选项组成。命令名是要执行的命令，参数是命令要操作的对象或参数，选项是用来控制命令行为的标志。

三、实验步骤

本实验采用了 openEuler 20.03 操作系统[①]（其他版本类似），可从 openEuler 官网获得。同时，采用 VMware Workstation 17 Pro 作为虚拟化软件，并用 FinalShell（也可采用 XShell）远程登录至虚拟机系统中进行操作（后续实验步骤会详细给出）。为顺利完成本实验，要求物理主机的性能满足：CPU 性能不要太差（i5 及以上更优）、内存 16GB 以上、硬盘可用空间 60GB

[①] https://repo.huaweicloud.com/openeuler/openEuler-20.03-LTS-SP2/ISO/x86_64/openEuler-20.03-LTS-SP2-x86_64-dvd.iso

以上，虚拟机中安装 openEuler 时，物理主机保证连接 Internet。

3.1 虚拟机软件安装

首先，下载安装包，网址为：https://www.vmware.com/cn/products/workstation-pro.html。

执行安装程序（以管理员方式运行）。开始安装时，界面如图 1.1 所示，紧接着根据图 1.2~图 1.4 的提示进行操作。

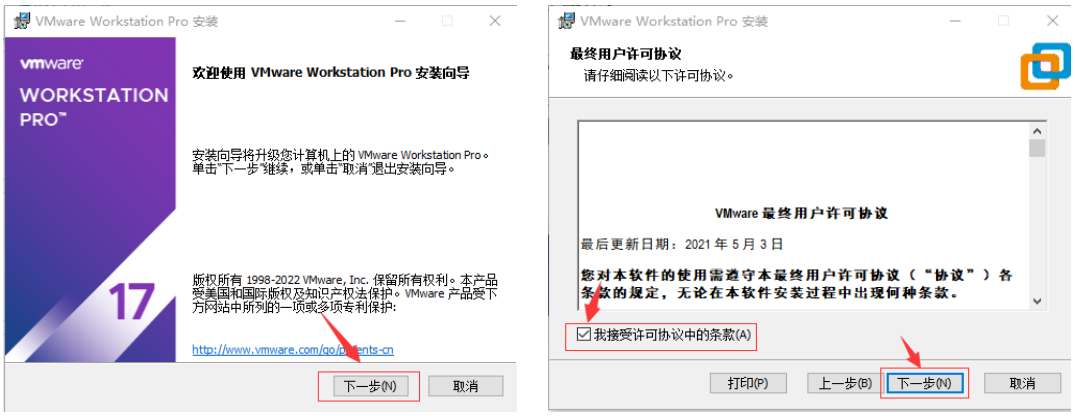


图 1.1　VMware Workstation 安装（I）



图 1.2　VMware Workstation 安装（II）

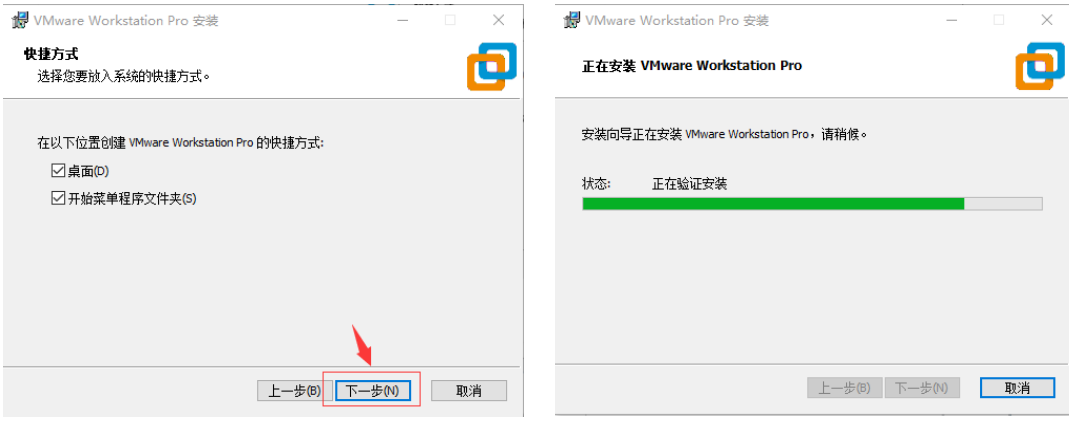


图 1.3　VMware Workstation 安装（III）

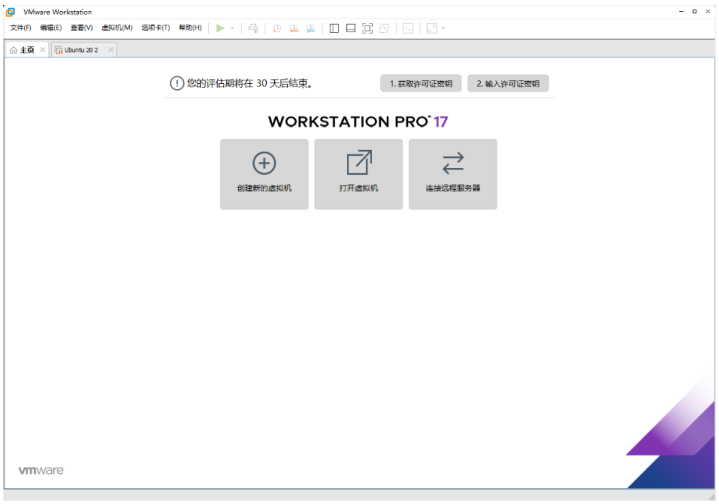


图 1.4　安装完成后虚拟机界面

3.2 openEuler 系统安装

详细的安装过程如下所示：

首先点击文件→新建虚拟机进入新建虚拟机向导：

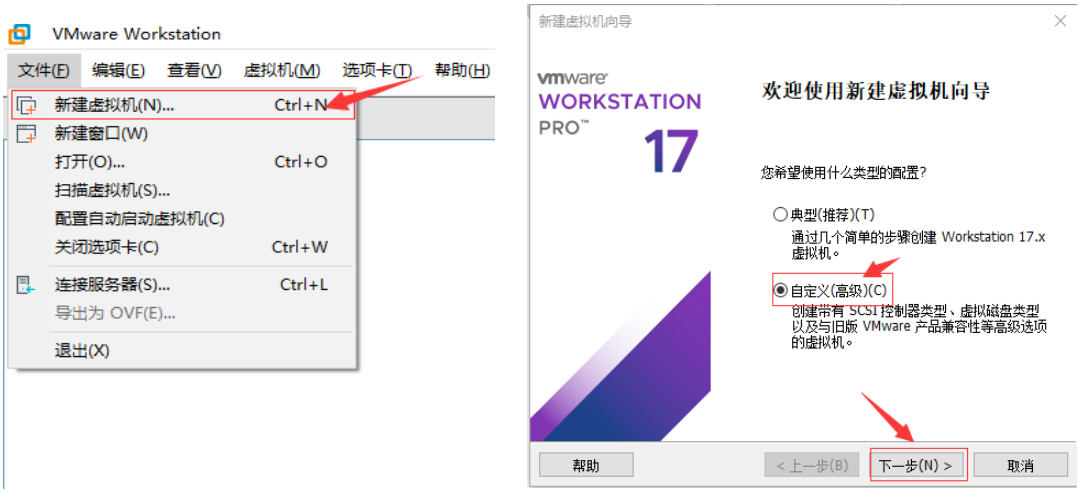


图 1.5　开始创建虚拟机

进入新建虚拟机向导后大部分操作都直接点下一步。

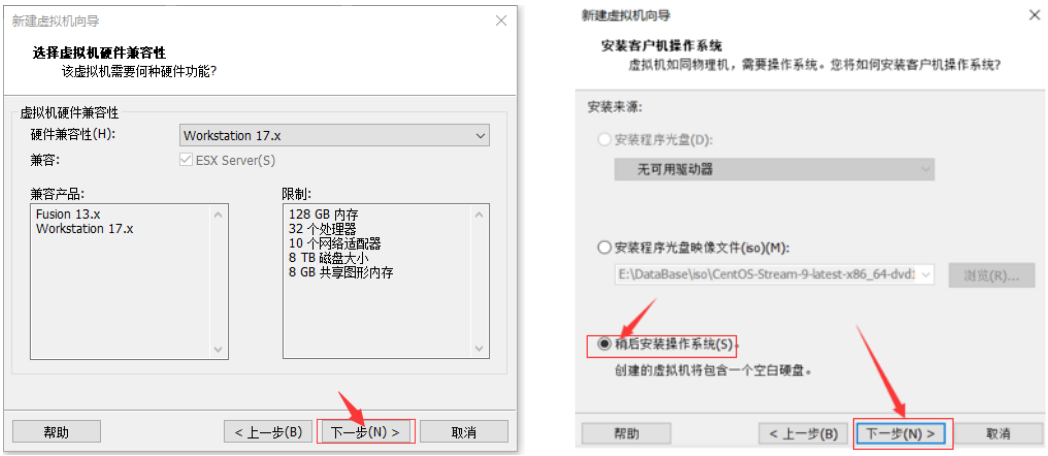


图 1.6　虚拟机硬件兼容和虚拟机操作系统安装

因为我们选择的 OpenEuler20.03 的内核为 Linux 4.x 内核，所以这一步选择相应内核并选定虚拟机在主机中的位置。

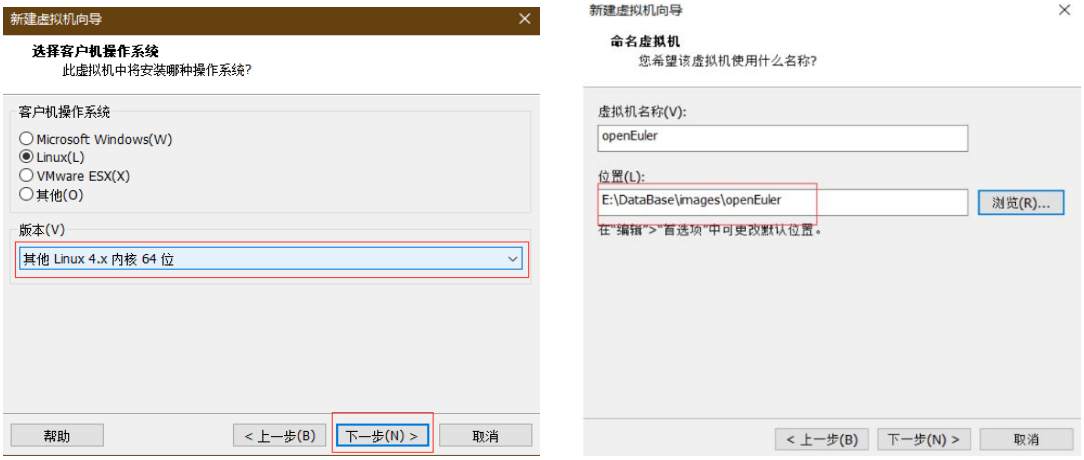


图 1.7　选择对应的操作系统内核以及选择虚拟机存放位置

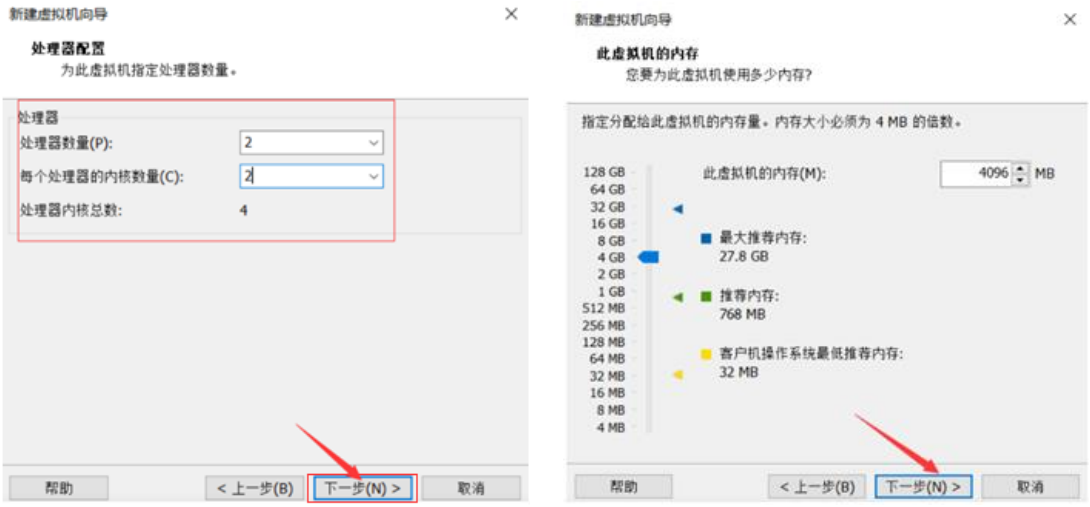


图 1.8　处理器配置以及内存分配

下面是网络类型设置以及 I/O 控制器设置(可以直接点击下一步)。



图 1.9　网络类型设置以及 I/O 控制器类型设置

磁盘类型设置：



图 1.10　磁盘类型设置



图 1.11　磁盘空间大小设置、磁盘文件名称设置

点击完成后出现图 1.12(右边)画面，点击编辑虚拟机设置：

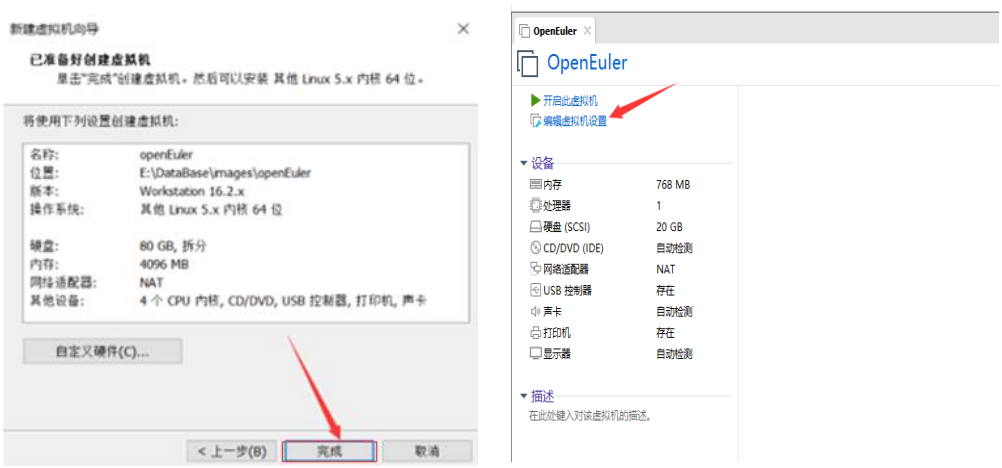


图 1.12　新建虚拟机向导完成

在虚拟机设置中点击 CD/DVD 使用镜像文件，使用镜像文件后，打开虚拟机。

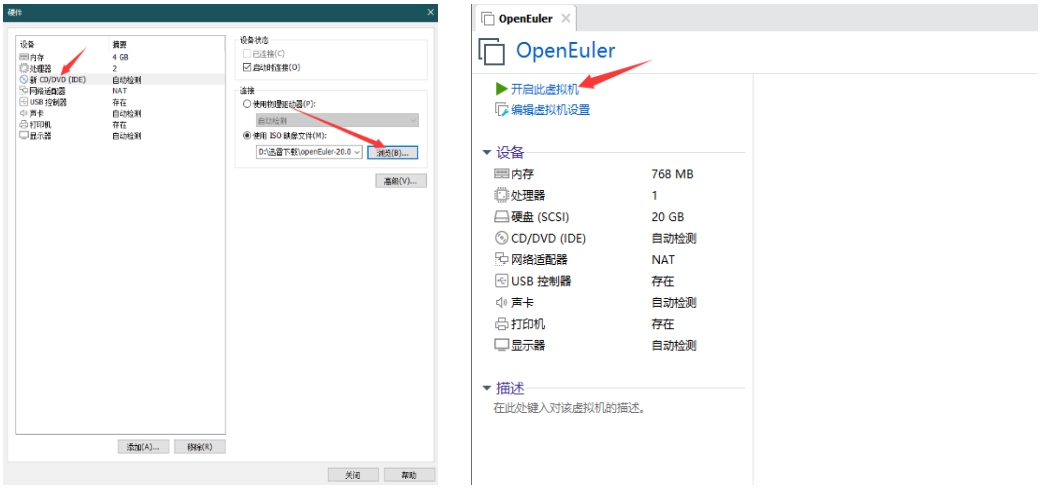


图 1.13　使用镜像文件

虚拟机设置完成后下面开始正式安装 openeuler 系统。

按 enter 键进入 Install openEuler 20.03-LTS 界面，等待初始化结束后出现图 1.14 右边画面，点击中文进行安装。



图 1.14　openEuler 安装界面

点击**安装目的地(D)**设置磁盘：



图 1.15　设置安装目的地

点击**网络和主机名(N)**设置网络选项：



图 1.16　设置网络和主机名

点击**根密码(R)**设置主机密码：



图 1.17　设置根密码

点击**开始安装**后等待安装自动完成，之后点击重启系统。



图 1.18　开始安装

重启虚拟机后会出现终端命令行界面，localhost(也就是初始账号)为 host，PassWord 为图 1.17 中自己设置的密码。

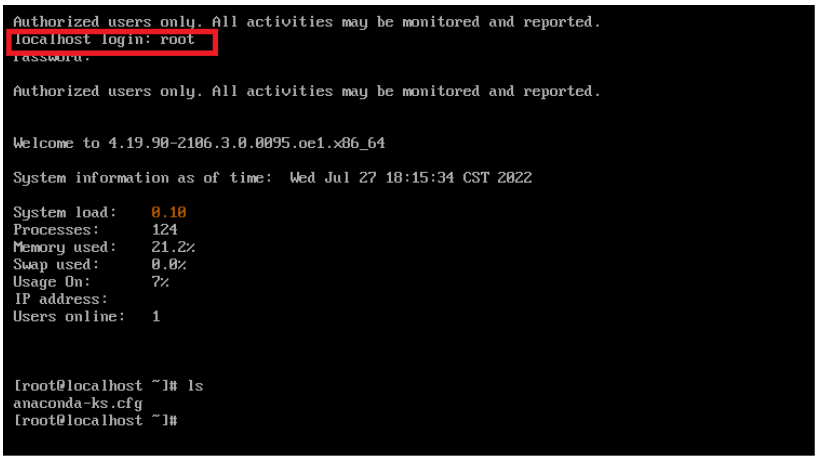


图 1.19　新内核安装重启后选择启动内核界面

3.3 FinalShell 软件安装

双击点击 finalshell_install.exe 文件，进入 finalshell 安装界面，点击**我接受**。

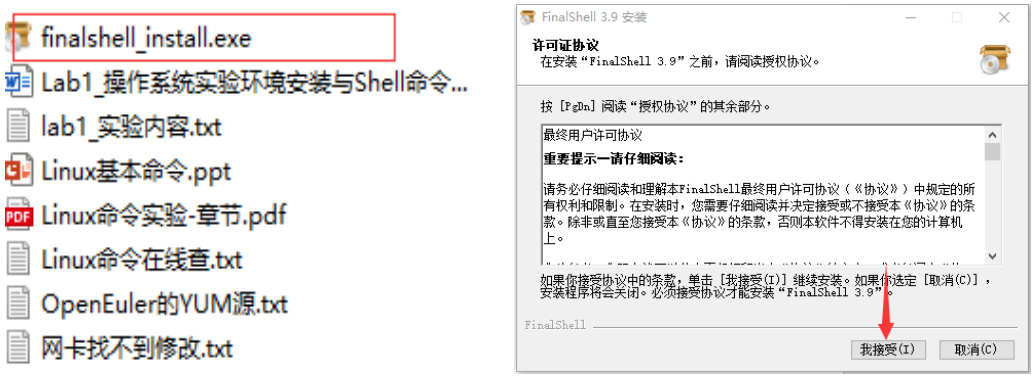


图 1.20　FinalShell 安装

选择安装的组件以及相应安装的文件夹位置后选择安装。



图 1.21　FinalShell 安装

选择安装的组件以及相应安装的文件夹位置后选择安装。



图 1.22　FinalShell 安装

等待安装完成后点击关闭，FinalShell 的安装结束。

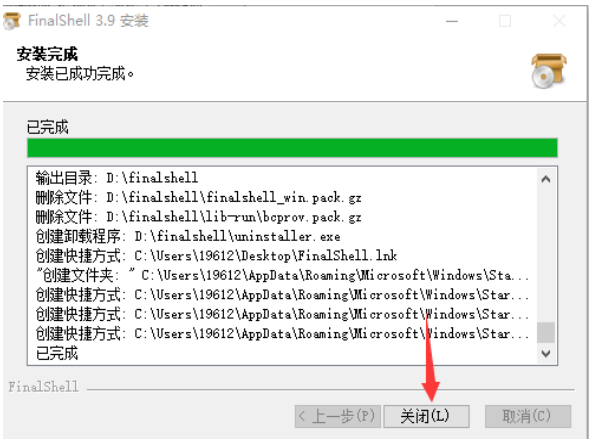


图 1.23　FinalShell 安装完毕

接下来连接我们的虚拟机和 FinalShell，在命令行中输入 ip addr 命令查看虚拟机的 ip，观察 ens33 端口的 ip，出现下面的情况则需我们修改端口文件进行 ip 的自动配置。

```
[root@localhost ~]# ip addr
1：lo…..
2：ens33:<BROADCAST,MULTICAST,UP,LOWE_UP>mtu 1500 qdisc fq_codel state UP group default qlen 1000 link/ether 00:0c:29:0e:c3:22 ff:ff:ff:ff:ff:ff
```

在命令行中输入：

```
[root@localhost ~]# vi /etc/sysconfig/network-scripts/ifcfg-ens33
```

进入 vi 界面后按下 i 进入 insert 模式，将“ONBOOT”后面的值设为 yes。

```
TYPE=Ethernet
PROXY_METHOD=none
BROWSER_ONLY=no
BOOTPROTO=dhcp
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
IPV6_ADDR_GEN_MODE=stable-privacy
NAME=ens33
UUID=ae86bd93-1a67-4bf9-8264-8fa2c98b8556
DEVICE=ens33
ONBOOT=yes
```

点击键盘上“Esc”键，输入“:wq”并回车，退出 vi。

之后在命令行中输入 reboot，重启观察到以下界面则 ip 配置成功。

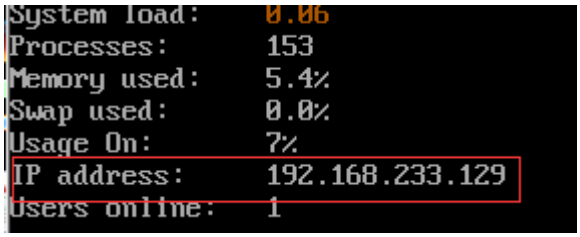


图 1.24　ip 配置

打开 FinalShell 软件点击左上角文件夹标志打开连接管理器，再点击左上角白色文件夹标志创建新的连接。

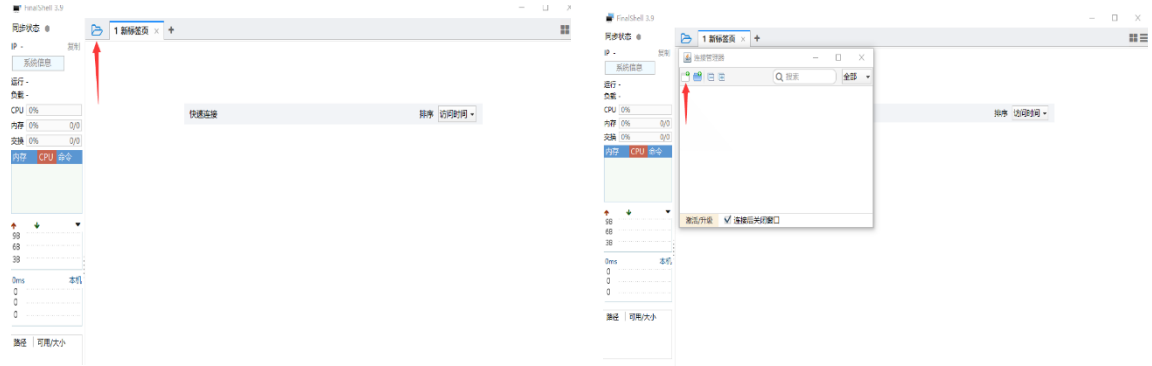


图 1.25　创建新的连接

下面开始新建连接，点击 SSH 连接(Linux)进入新建连接界面，在名称处填入连接名称(可随意)，主机处填入 ip(每个主机根据图 1.24 的不同结果进行填入)，用户名处填入 root，密码处填入自己设置的虚拟机密码，完成后点击确定。

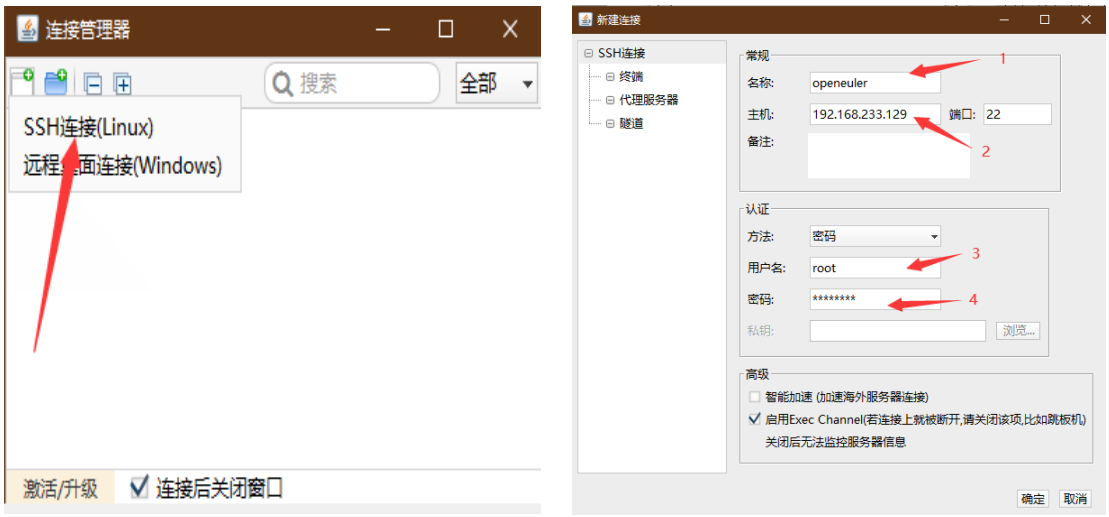


图 1.26　创建新的连接

完成创建后连接出现新建连接，右键点击连接，出现图 1.27 右边界面表示连接成功。

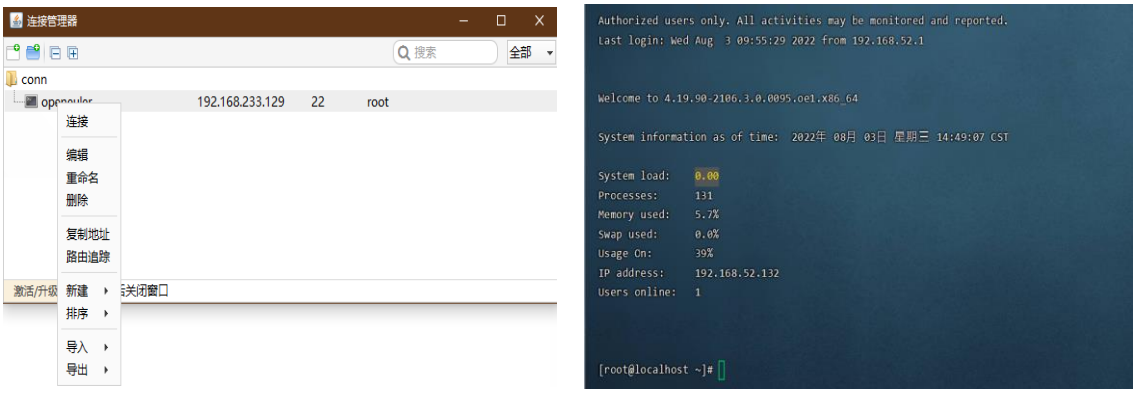


图 1.27　连接成功

3.3 Shell 常用命令使用

➢ 目录操作

（1）执行 ls 命令，查看目录。

```
[root@localhost ~]# ls
anaconda-ks.cfg    huchi2.c        jingcheng2.c    server.c    test    to_be_locked.txt
a.out              huchi.c         pipe            signal      test1
client             jincheng.c      pipe.c          signal.c    test2
client.c           jingcheng1.c    server          ssbm.c      test3
```

（2）执行 mkdir [OPTION]…[DIRECTORY]…，创建一个新目录，再执行 ls 命令进行查看，发现此时增加了一个目录 lic。

```
[root@localhost ~]# mkdir lic
[root@localhost ~]# ls
anaconda-ks.cfg    huchi.c         pipe          signal.c    test3
a.out              jincheng.c      pipe.c        ssbm.c      to_be_locked.txt
client             jingcheng1.c    server        test
client.c           jingcheng2.c    server.c      test1
huchi2.c           lic             signal        test2
```

（3）执行 pwd 命令，查看当前所在目录的全路径。

```
[root@localhost ~]# pwd
/root
```

（4）执行 rm [OPTION]... [FILE]…，删除一个文件或者目录，再执行 ls 命令查看。

```
[root@localhost ~]# rm -rf lic
[root@localhost ~]# ls
anaconda-ks.cfg    huchi2.c        jingcheng2.c    server.c    test    to_be_locked.txt
a.out              huchi.c         pipe            signal      test1
client             jincheng.c      pipe.c          signal.c    test2
client.c           jingcheng1.c    server          ssbm.c      test3
```

（5）执行 cd [dirName]，切换当前工作目录。

```
[root@localhost ~]# mkdir root1
[root@localhost ~]# cd root1
[root@localhost root1]# pwd
/root/root1
```

- **文件操作**

（1）执行 which 命令，查找 Linux 命令位置。

```
[root@localhost ~]# which mkdir
/usr/bin/mkdir
```

（2）执行 touch [OPTION]... [FILE]...创建一个文件，再执行 echo “内容” >[文件名]和 echo “内容” >>[文件名]向文件中写入和追加内容，最后执行 cat [文件名]查看文件内容。

```
[root@localhost ~]# touch osc.txt
[root@localhost ~]# echo "This is a test.">osc.txt
[root@localhost ~]# echo "This is the addition">>osc.txt
[root@localhost ~]# cat osc.txt
This is a test.
This is the addition
```

（3）执行 find 命令，查找文件。

```
[root@localhost ~]# find -name "o*"
./osc.txt
```

（4）执行 grep 命令，在文件中查找并显示包含指定字符串的行。

```
[root@localhost ~]# grep "This" osc.txt
This is a test.
This is the addition
```

（5）执行 wc 命令，统计文件中的单词数量等信息。

```
[root@localhost ~]# grep "This" osc.txt
This is a test.
This is the addition
```

- **系统询问与改变权限**

（1）执行 who 命令，查看系统中的使用者。

```
[root@localhost ~]# who
root     tty1         2021-12-01 13:06
root     pts/0        2021-12-01 13:07 (192.168.52.1)
root     pts/1        2021-12-01 13:07 (192.168.52.1)
```

- **进程操作**

（1）执行 ps[OPTION]命令，显示当前进程的状态。

```
[root@localhost ~]# ps
```

```
  PID TTY          TIME CMD
 1446 pts/0    00:00:00 bash
169973 pts/0    00:00:00 ps
```

（2）在当前目录下，执行 gzip -d 命令，解压.gz 文件

```
[root@localhost ~]# gzip -d osc.txt
[root@localhost ~]# ls
anaconda-ks.cfg  huchi2.c     jingcheng2.c  server.c  test      to_be_locked.txt
a.out            huchi.c      pipe          signal    test1
client           jincheng.c   pipe.c        signal.c  test2
client.c         jingcheng1.c server        ssbm.c    test3     osc.txt
```

- **其他**

（1）执行 df 命令，显示目前 Linux 系统上的文件系统磁盘使用情况统计。

```
[root@localhost ~]# df
文件系统                    1K-块     已用      可用  已用% 挂载点
devtmpfs                 3791648        0   3791648    0% /dev
tmpfs                    3808408        0   3808408    0% /dev/shm
tmpfs                    3808408    42008   3766400    2% /run
tmpfs                    3808408        0   3808408    0% /sys/fs/cgroup
/dev/mapper/openeuler-root 17410832 2575520 13927844   16% /
tmpfs                    3808408        0   3808408    0% /tmp
/dev/sda1                 999320   128292    802216   14% /boot
tmpfs                     761680        0    761680    0% /run/user/0
```

（2）执行 free 命令，显示内存状态。

```
[root@localhost ~]# free
               total        used        free      shared  buff/cache   available
Mem:         7616820      199364     7080308       50196      337148     7019464
Swap:        2097148           0     2097148
```

（3）执行 date 命令。

```
[root@localhost ~]# date
2023 年  4 月  18 日 星期二 13:49:45 CST
```

（4）执行 cal 命令。

```
[root@localhost ~]# cal
      四月 2023
一  二  三  四  五  六  日
                1   2   3   4   5
 6   7   8   9  10  11  12
13  14  15  16  17  18  19
20  21  22  23  24  25  26
27  28  29  30
```

四、实验讨论

1、为什么选择 openEuler 作为操作系统实验的基础平台？有哪些优点和缺点？

2、在实验中使用 FinalShell 时，你遇到了哪些问题？你是如何解决这些问题的？

3、除了第三节所提到的 Shell 命令，还有哪些常用的、便利的 Shell 命令？